

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Лабораторная работа №2

Реализация микросервисов

по дисциплине «Программирование серверной логики»

Вариант: Сайт для поиска работы (Job Search Platform)

Студент: Гельм Даниил

Группа: К3340 / БР1.1

Санкт-Петербург
2026

Содержание

1. Цель работы
2. Исходные данные
3. Реализованная архитектура
4. Разделение базы данных
5. Межсервисное взаимодействие
6. Структура проекта и запуск
7. Тестирование
8. Выводы
9. Приложение А. Код диаграммы

1. Цель работы

Целью лабораторной работы №2 является реализация микросервисной архитектуры для бэкенд-приложения «Сайт для поиска работы» на основе технического дизайна, выполненного в ДЗ4.

Необходимо разделить монолитную реализацию ЛР1 (labs/lab1) на независимые микросервисы с принципом database-per-service, обеспечить межсервисное взаимодействие и сохранить работоспособность публичного API для клиента.

2. Исходные данные

Исходный монолит — : одно Express-приложение, TypeORM, одна база PostgreSQL job_search. Публичное API: /api/v1/* на порту 3000.

Проектирование разделения выполнено в ДЗ4: 7 доменных сервисов, API Gateway, отдельная БД на сервис, синхронное HTTP-взаимодействие между сервисами.

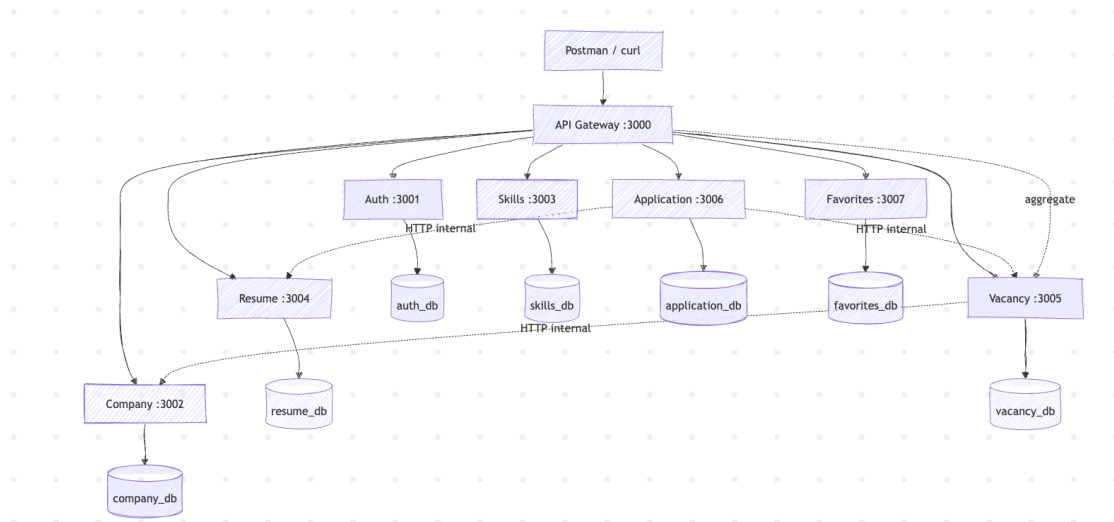
Стек реализации ЛР2: Node.js, TypeScript, Express, TypeORM, PostgreSQL, Docker Compose.

3. Реализованная архитектура

Компонент	Порт	Папка	БД
API Gateway	3000	gateway/	—
Auth Service	3001	services/auth/	auth_db
Company Service	3002	services/company/	company_db
Skills Service	3003	services/skills/	skills_db
Resume Service	3004	services/resume/	resume_db
Vacancy Service	3005	services/vacancy/	vacancy_db
Application Service	3006	services/application/	application_db
Favorites Service	3007	services/favorites/	favorites_db

Клиент (Postman) обращается только к API Gateway. Gateway проксирует запросы в соответствующий микросервис. Для GET /favorites Gateway агрегирует данные из Favorites Service и Vacancy Service.

3.1. Диаграмма архитектуры



Что вставить: Код Mermaid в Приложении А → mermaid.live → Export PNG → вставить сюда. Или скриншот структуры папок lab2 из VS Code.

4. Разделение базы данных

Каждый микросервис использует собственную базу PostgreSQL (database-per-service). Связи между доменами — логические ссылки по ID, без FK между разными базами.

Сервис	БД	Таблицы
Auth	auth_db	users
Company	company db	companies
Skills	skills_db	skills
Resume	resume_db	resumes
Vacancy	vacancy_db	vacancies
Application	application_db	applications
Favorites	favorites_db	favorite vacancies

```
Last login: Sat Jun 6 14:12:29 on ttys009
[daniilgelm@m1 ~ % docker exec -it lab2-postgres-1 psql -U postgres -c "\l"
                                List of databases
  Name      | Owner   | Encoding | Locale Provider | Collate  | Ctype    | ICU Locale | ICU Rules | Access privileges
-----+-----+-----+-----+-----+-----+-----+-----+-----
application_db | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
auth_db        | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
company_db     | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
favorites_db   | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
postgres       | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
resume_db      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
skills_db      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           |
template0      | postgres | UTF8     | libc            | en_US.utf8 | en_US.utf8 |             |           | =c/postgres +
               |         |         |         |         |         |             | postgres=CtC/postgres
--More--
```

5. Межсервисное взаимодействие

Между микросервисами используется синхронное HTTP-взаимодействие (internal API). Аутентификация service-to-service — заголовок X-Service-Token.

Сценарий	Кто вызывает	Кого	Зачем
Создание вакансии	Vacancy Service	Company Service	Проверка companyId и прав
Создание отклика	Application Service	Vacancy + Resume	Проверка существования и владельца
GET /favorites	API Gateway	Favorites + Vacancy	IDs → batch-запрос вакансий

Общий код (типы, JWT, HTTP-клиент для internal-вызовов) вынесен в каталог shared/.

6. Структура проекта и запуск

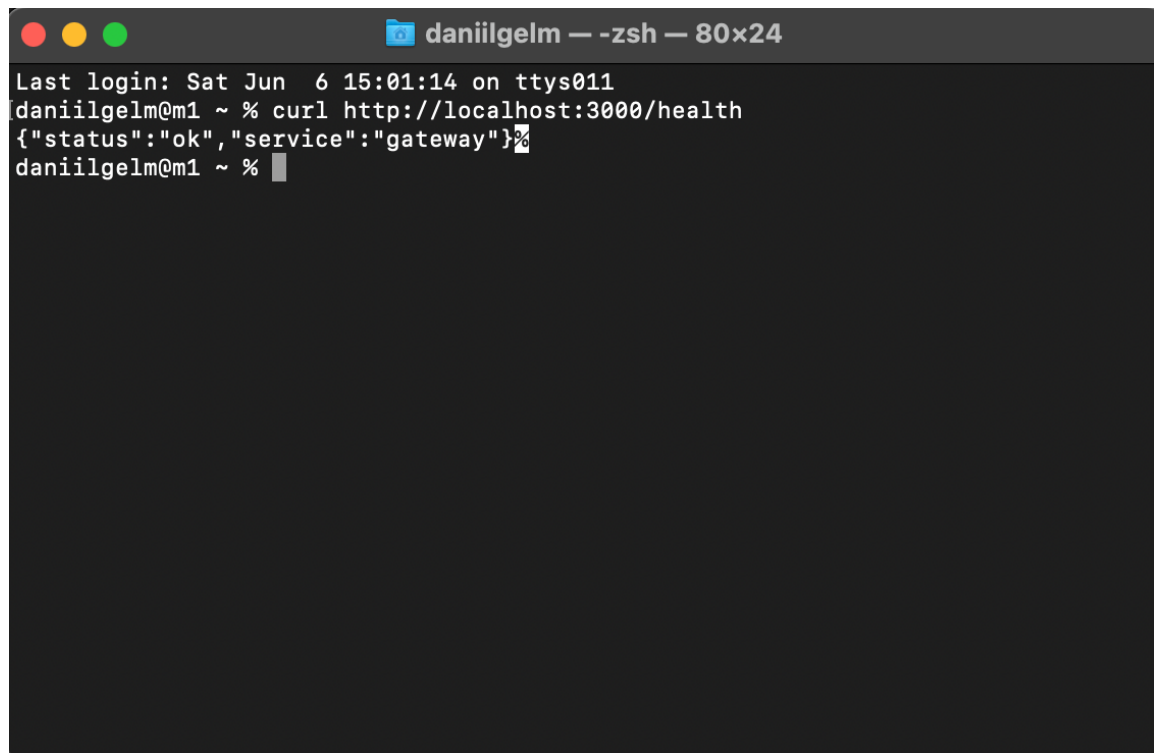
Корневая структура labs/lab2:

```
lab2/
  gateway/           - API Gateway
  services/
    auth/ company/ skills/ resume/
    vacancy/ application/ favorites/
  shared/           - общий код
  docker-compose.yml - PostgreSQL (7 БД)
  scripts/setup.sh  - установка зависимостей
```

6.1. Команды запуска

```
cd labs/lab2
npm run setup      # первый раз
npm run docker:up  # PostgreSQL :5434
npm run dev        # Gateway + 7 сервисов
```

6.2. Проверка health

A terminal window titled 'daniilgelm — zsh — 80x24' with standard macOS window controls (red, yellow, green buttons). The terminal shows the following text:

```
Last login: Sat Jun 6 15:01:14 on ttys011
daniilgelm@m1 ~ % curl http://localhost:3000/health
{"status":"ok","service":"gateway"}%
daniilgelm@m1 ~ % █
```

7. Тестирование

Тестирование выполнено средствами Postman. Использована коллекция из ДЗЗ (комплексный сценарий поиска работы). Запросы направляются через API Gateway на <http://localhost:3000/api/v1>.

7.1. Файлы Postman

Файл	Назначение
homeworks/hw3/DZ3_Job_Search_Scenario.postman_collection.json	Коллекция запросов
homeworks/hw3/DZ3_Local.postman_environment.json	Окружение (host, baseUrl)

Параметры окружения для ЛР2:

Переменная	Значение
host	http://localhost:3000
baseUrl	http://localhost:3000/api/v1

7.2. Комплексный сценарий

Выполнен прогон папки «Комплексный сценарий» (16 запросов): регистрация работодателя, создание компании и вакансии, регистрация соискателя, резюме, поиск вакансий, отклик, избранное, смена статуса отклика.

Даниил Даниил's Workspace

Search

Invite Upgrade

DZ3 — сценарий (поиск работы) — Run results

+ New Run Share

Ran today at 02:57:11 PM

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	DZ3 Local (Job Search)	1	3s 278ms	31	0	-

All 31 Passed 31 Failed 0 Skipped 0 Errors 0 Console log

List Grid

PASS Статус sent

POST Комплексный сценарий > 13 Add favorite
http://localhost:3000/api/v1/vacancies/2/favorite201 • 77 ms • 280 B2

PASS Избранное 201

PASS ok true

GET Комплексный сценарий > 14 List favorites
http://localhost:3000/api/v1/favorites200 • 15 ms • 561 B2

PASS Список избранного 200

PASS Вакансия в избранном

GET Комплексный сценарий > 15 My applications
http://localhost:3000/api/v1/applications/my200 • 22 ms • 421 B2

PASS Мои отклики 200

PASS Есть отклик

PATCH Комплексный сценарий > 16 Employer update application status
http://localhost:3000/api/v1/applications/2/status200 • 25 ms • 421 B2

PASS Обновление статуса 200

PASS Статус viewed

COLLECTIONS

DZ3 — сценарий (поиск работы)

Комплексный сценарий

GET 01 Health

POST 02 Register employer

POST 03 Login employer

POST 04 Create company

POST 05 Create vacancy

POST 06 Register applicant

POST 07 Login applicant

GET 08 Me applicant

POST 09 Create resume

GET 10 List vacancies filter

GET 11 List companies

POST 12 Apply to vacancy

POST 13 Add favorite

GET 14 List favorites

GET 15 My applications

PATCH 16 Employer update application status

New Collection

ENVIRONMENTS

SPECS

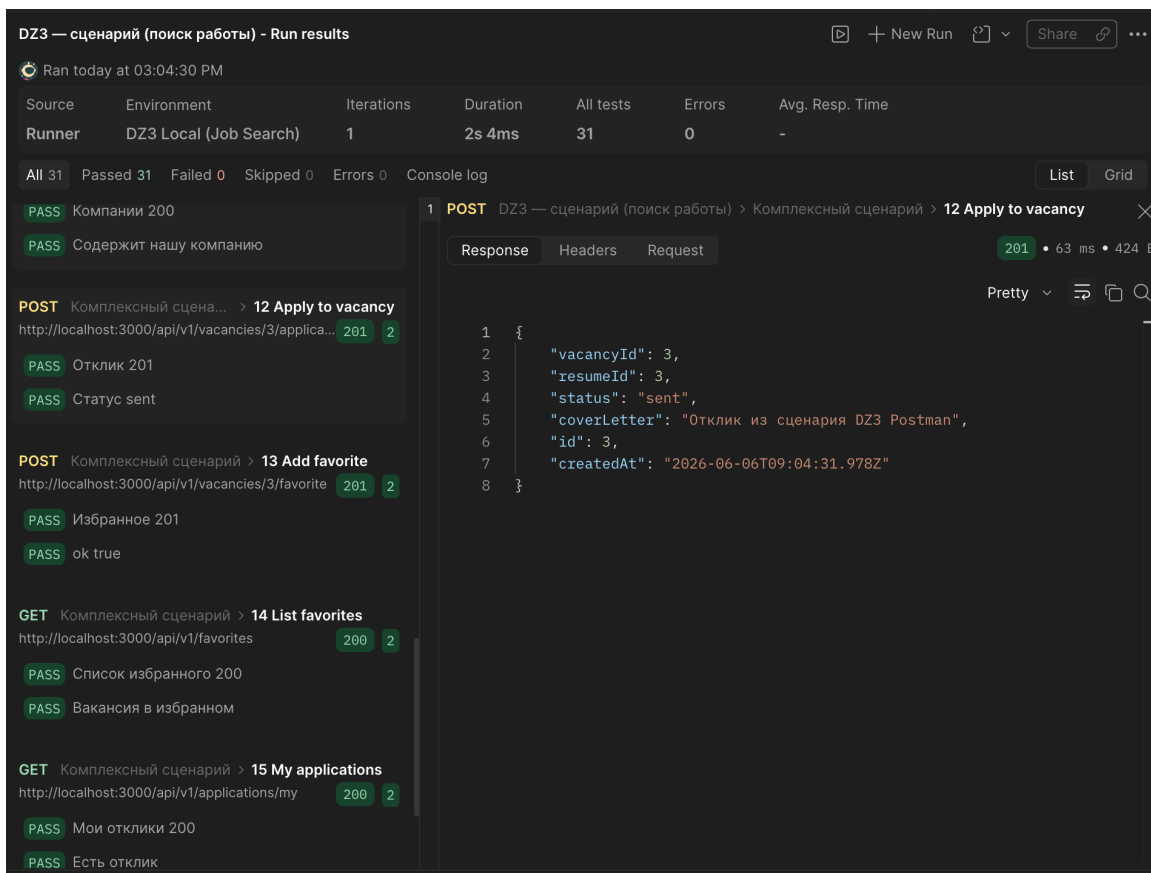
MOCKS

FLOWES

lab2

Console Terminal

Globals Vault Tools



8. Выводы

В ходе выполнения ЛР2 монолит Job Search Platform разделён на 7 микросервисов и API Gateway с изолированными базами данных. Публичный контракт `/api/v1` сохранён; комплексный Postman-сценарий успешно выполняется через Gateway.

Межсервисное взаимодействие реализовано синхронными HTTP-вызовами. Приложение готово к дальнейшему развитию (ДЗ5 — RabbitMQ, ЛР3 — Docker-контейнеры сервисов).